Bryan Emer

DSC 650

Final Project

End-to-End Big Data Pipeline: Spotify Tracks Popularity Prediction

Introduction

The objective of this final project was to design and implement a complete end-to-end data pipeline integrating six major components of the Hadoop and Spark ecosystem: Apache NiFi, HDFS, YARN, Apache Hive, Apache HBase, and Apache Spark with MLlib. The pipeline was built inside a Docker-based environment running on a Google Cloud virtual machine with 8 vCPUs and 16 GB of RAM.

The pipeline follows a logical data flow: NiFi ingests a raw CSV dataset from GitHub and writes it to HDFS for distributed storage. From HDFS, the data is loaded into a Hive managed table for structured querying and schema enforcement. A PySpark MLlib job then reads from Hive, trains a Random Forest regression model to predict track popularity from audio features, evaluates model performance, and writes the resulting metrics to an HBase NoSQL table for persistence. The final HBase scan confirms that all components of the pipeline executed successfully in sequence.

This project demonstrates the ability to work across the full Hadoop ecosystem in a realistic distributed computing environment, including diagnosing and resolving real-world errors encountered during execution.

Dataset

Dataset Name: Spotify Tracks Dataset

Source: Kaggle - maharshipandya/spotify-tracks-dataset

GitHub Direct URL:
https://raw.githubusercontent.com/bemer303/bigdata/refs/heads/main/spotify_dataset.csv

File Name: spotify_tracks.csv

Row Count: ~114,000 tracks

Format: CSV with header row

The dataset contains audio feature metadata for approximately 114,000 Spotify tracks across 114 genres. Each row represents one track and includes the following 21 columns:

Column	Type	Description
row_num	STRING	Unnamed Kaggle index column
track_id	STRING	Spotify track identifier
artists	STRING	Artist name(s)
album_name	STRING	Album title
track_name	STRING	Track title
popularity	INT	Popularity score 0–100 (target variable)
duration_ms	INT	Track length in milliseconds
explicit	INT	Explicit content flag (0/1)
danceability	FLOAT	Danceability score 0.0–1.0
energy	FLOAT	Energy level 0.0–1.0
track_key	INT	Musical key (0–11)
loudness	FLOAT	Overall loudness in dB
mode	INT	Major (1) or minor (0)
speechiness	FLOAT	Presence of spoken words
acousticness	FLOAT	Acoustic confidence 0.0–1.0
instrumentalness	FLOAT	Instrumentalness 0.0–1.0
liveness	FLOAT	Presence of live audience

Column	Type	Description
valence	FLOAT	Musical positivity 0.0–1.0
tempo	FLOAT	Estimated BPM
time_signature	INT	Estimated beats per bar
track_genre	STRING	Genre label

This dataset was chosen because it contains a rich set of numeric audio features suitable for regression modeling, has enough rows (114K) to meaningfully train and evaluate an MLlib model, and is domain-relevant, making the results interpretable in musical terms.

Pipeline Overview

The pipeline integrates all six required components in the following sequence:

GitHub (CSV) → NiFi → HDFS → Hive → Spark MLlib (YARN) → HBase

Step 1 - NiFi (Data Ingestion):

Apache NiFi was used to download the Spotify CSV from a raw GitHub URL and write it to HDFS. The provided Final_Project.json template was imported into NiFi and configured with a Parameter Context containing four variables: DOWNLOAD FILE URL, FILENAME, HDFS WRITE DIRECTORY, and USER. The flow uses three processors: InvokeHTTP to download the file, UpdateAttribute to rename it, and PutHDFS to write it to /user/spotify in HDFS.

Step 2 - HDFS (Distributed Storage):

HDFS served as the central storage layer. The file was confirmed with hdfs dfs -ls /user/spotify, showing spotify_tracks.csv successfully written. HDFS provides fault-tolerant, distributed file storage that allows both Hive and Spark to access the data without copying it to local disk.

Step 3 - Hive (Data Warehousing):

A managed Hive table named spotify_tracks was created in a dedicated database spotify_db. The schema was designed to match the CSV's 21 columns including the unnamed index column. Data was loaded using LOAD DATA INPATH. Aggregation queries were run to verify that numeric columns were correctly populated across all rows.

Step 4 - YARN (Resource Management):

YARN served as the resource manager for the Spark job. All spark-submit commands used --master yarn, causing YARN to allocate containers across the master and worker nodes for distributed execution. YARN managed memory allocation and task scheduling throughout the MLlib training process.

Step 5 - Spark + MLlib (Machine Learning):

A PySpark script (spotify_ml.py) was submitted via spark-submit. The script reads data from the Hive table using SparkSQL, assembles 13 audio feature columns into a feature vector, trains a Random Forest Regressor to predict popularity, and evaluates the model using RMSE, R^2, and MAE metrics.

Step 6 - HBase (NoSQL Metrics Storage):

After model evaluation, the PySpark script wrote all metrics and model metadata to an HBase table named spotify_ml_metrics using the happybase Python library over the Thrift protocol. A final scan command in the HBase shell confirmed all eight columns were successfully written.

Issues Encountered

Issue 1 - YARN Memory Threshold Exceeded

Error: Required executor memory (2048 MB), overhead (384 MB) is above the max threshold (1536 MB)

Cause: The initial spark-submit specified --executor-memory 2g and --driver-memory 2g. The YARN cluster's per-container memory cap was 1536 MB, and 2048 + 384 MB overhead exceeded this limit.

Resolution: Reduced both flags to 1g (1024 MB + 384 MB overhead = 1408 MB), which fit within the 1536 MB cap. At this scale, 1g was sufficient for the 114K-row dataset

Issue 2 - StandardScaler Empty Partition Error

Error: java.lang.IllegalArgumentException: requirement failed: Nothing has been added to this summarizer

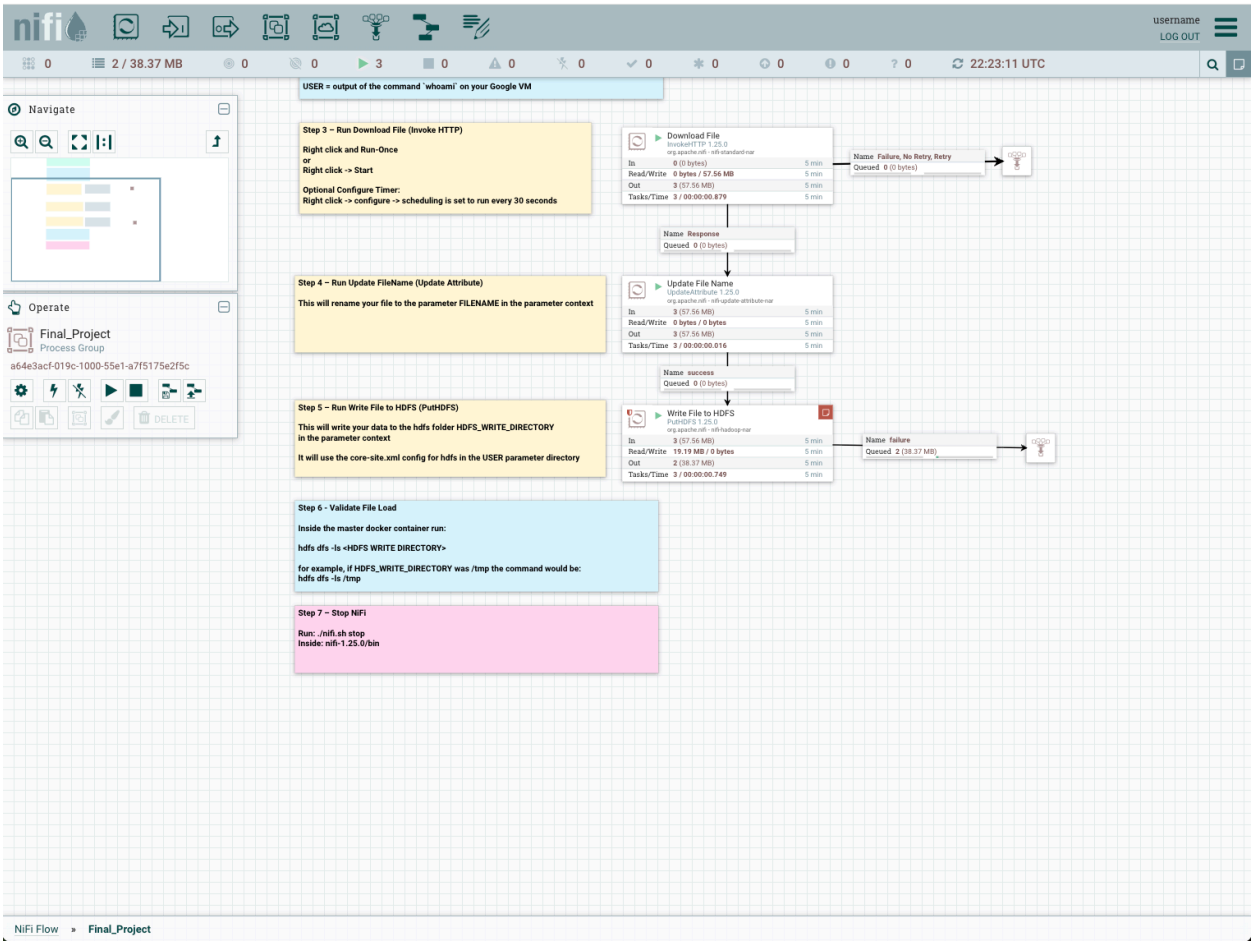
Cause: The pipeline originally included a StandardScaler stage between the VectorAssembler and RandomForestRegressor. When randomSplit divided the data, some partitions were empty, causing the StandardScaler's internal statistics summarizer to fail with nothing to process.

Resolution: Removed the StandardScaler entirely. Random Forest is a tree-based algorithm and does not require feature scaling. Unlike gradient-based or distance-based models, it is invariant to feature magnitude. Removing the scaler also simplified the pipeline without any loss of model quality. Additionally, df.repartition(4) was added to ensure balanced data distribution across partitions.

Screenshots

Objective 1 –

NiFi Flow Design, NiFi Flow Running -



HDFS verification -

```
bryanemer@dsc650-bigdata:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase$ docker-compose exec master bash
bash-5.0# hdfs dfs -ls /user/spotify
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/program/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/tez/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/program/hive/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
2026-02-28 22:23:15,292 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 1 items
-rw-r--r--   3 bryanemer supergroup   20118244 2026-02-28 22:21 /user/spotify/spotify_dataset.csv
bash-5.0#
```

The NiFi pipeline uses three processors to ingest the Spotify Tracks dataset into HDFS. InvokeHTTP issues an HTTP GET to the raw GitHub URL and retrieves the CSV as a FlowFile. UpdateAttribute renames it to spotify_tracks.csv for consistent downstream references. PutHDFS writes the file to the /user/spotify HDFS directory using the Hadoop core-site.xml configuration. This mirrors real-world ingestion patterns where NiFi acts as a lightweight ETL broker between external sources and distributed storage.

Objective 2

Create Table statement -

```
hive> LOAD DATA INPATH '/user/spotify/spotify_dataset.csv' INTO TABLE spotify_tracks;
Loading data to table spotify_db.spotify_tracks
OK
Time taken: 1.638 seconds
hive>
```

Data successfully loaded into Hive -

```
hive> SELECT COUNT(*) AS total_tracks FROM spotify_tracks;
2026-02-28 22:39:53,957 INFO  [ffe378d9-3b7f-4aa1-8744-1cc60ccdc032 main] reducesink.VectorReduceSinkEmptyKeyOperator: VectorReduceSinkEmptyKeyOperator constructor vectorReduceSinkI
rg.apache.hadoop.hive.ql.plan.VectorReduceSinkInfo@63d66761
Query ID = root_20260228223949_185bb464-d0ac-4657-a72e-051128fcb3a1
Total jobs = 1
Launching Job 1 out of 1
Tez session was closed. Reopening...
2026-02-28 22:39:55,002 INFO  [ffe378d9-3b7f-4aa1-8744-1cc60ccdc032 main] client.RMProxy: Connecting to ResourceManager at master/172.28.1.1:8032
Session re-established.
Session re-established.
Status: Running (Executing on YARN cluster with App id application_1772315250539_0008)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 .......... container     SUCCEEDED      1          1        0        0       0       0
Reducer 2 ...... container     SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 6.88 s
--------------------------------------------------------------------------------
OK
114000
Time taken: 20.248 seconds, Fetched: 1 row(s)
hive>
```

Aggregation query -

```
hive> SELECT track_genre,
    >        COUNT(*) AS num_tracks,
    >        ROUND(AVG(popularity), 2) AS avg_popularity,
    >        ROUND(AVG(danceability), 3) AS avg_danceability
    > FROM spotify_tracks
    > GROUP BY track_genre
    > ORDER BY avg_popularity DESC
    > LIMIT 10;
2026-03-01 00:10:23,028 INFO  [562678c8-9d21-41ed-a259-7ed6cc225611 main] reducesink.VectorReduceSinkObjectHashOperator: VectorReduceSinkObjectHashOperator constructor vectorReduceSinkIn
fo org.apache.hadoop.hive.ql.plan.VectorReduceSinkInfo@66682e8f
Query ID = root_20260301001015_29f8346a-a8d8-4e1f-b44e-7d7e4f091e0b
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1772315250539_0020)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 .......... container     SUCCEEDED      1          1        0        0       0       0
Reducer 2 ...... container     SUCCEEDED      1          1        0        0       0       0
Reducer 3 ...... container     SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 03/03  [==========================>>] 100%  ELAPSED TIME: 9.69 s
--------------------------------------------------------------------------------
OK
4       4666    33673.75        NULL
pop-film        990     59.27   0.597
k-pop   967     56.69   0.649
chill   974     53.53   0.665
sad     946     52.47   0.694
grunge  970     49.64   0.458
indian  987     49.57   0.592
anime   986     48.82   0.538
sertanejo       886     47.82   0.592
emo     936     47.8    0.601
Time taken: 22.074 seconds, Fetched: 10 row(s)
hive>
```

The Hive table spotify_tracks is designed as a managed TEXTFILE table with comma-delimited fields matching the Spotify dataset's CSV schema. Most audio feature columns (danceability, energy, tempo, etc.) are typed as FLOAT to preserve precision for ML modeling, while categorical fields like track_genre and artists use STRING. The skip.header.line.count table property ensures the CSV header row is ignored during loading. The aggregation query groups tracks by genre and computes average popularity and danceability, confirming that the schema is correctly mapped and data is fully populated.

Objective 3

Pip3 install output -

```
bryanemer@dsc650-bigdata:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase$ docker-compose exec worker1 bash
bash-5.0# pip3 install numpy happybase
Collecting numpy
  Downloading numpy-1.21.6.zip (10.3 MB)
     |████████████████████████████████| 10.3 MB 5.7 MB/s
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
    Preparing wheel metadata ... done
Collecting happybase
  Downloading happybase-1.3.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources
  Downloading importlib_resources-5.12.0-py3-none-any.whl (36 kB)
Collecting six
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Collecting thriftpy2>=0.4
  Downloading thriftpy2-0.6.0.tar.gz (996 kB)
     |████████████████████████████████| 996 kB 39.2 MB/s
  Installing build dependencies ... done
  WARNING: Missing build requirements in pyproject.toml for thriftpy2>=0.4 from https://files.pythonhosted.org/packages/cc/7c/bc038fb868b6b5474101197d42866fbc61839c4f739816587a748ac
/thriftpy2-0.6.0.tar.gz#sha256=621c263f99274d51a9a1deecf301845f1408d497bdafed682db6155132a99cf4 (from happybase).
  WARNING: The project does not specify a build backend, and pip cannot fall back to setuptools without 'wheel'.
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
    Preparing wheel metadata ... done
Collecting zipp>=3.1.0; python_version < "3.10"
  Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Collecting typing-extensions>=3.7.4; python_version < "3.8"
  Downloading typing_extensions-4.7.1-py3-none-any.whl (33 kB)
Collecting ply<4.0,>=3.4
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
     |████████████████████████████████| 49 kB 6.8 MB/s
Building wheels for collected packages: numpy, thriftpy2
  Building wheel for numpy (PEP 517) ... done
  Created wheel for numpy: filename=numpy-1.21.6-cp37-cp37m-linux_x86_64.whl size=16644582 sha256=547207d58b8f07a24f5ceae5b6fa0fa36bbd57983fee81808777145ebb9bf308
  Stored in directory: /root/.cache/pip/wheels/4e/7e/9e/0fde042ccff2493994076dac9c3fbd78feb444c3bd94eb386a
  Building wheel for thriftpy2 (PEP 517) ... done
  Created wheel for thriftpy2: filename=thriftpy2-0.6.0-cp37-cp37m-linux_x86_64.whl size=1479782 sha256=477d13a97b1a9b8cbbbac65530a8e3837f62e0fbc0c6e67527a274985d8a209f
  Stored in directory: /root/.cache/pip/wheels/ce/e1/d3/38770f98821c966d3ad3bb392293b6e0bc189e89580338db63
Successfully built numpy thriftpy2
Installing collected packages: numpy, zipp, importlib-resources, six, typing-extensions, ply, thriftpy2, happybase
Successfully installed happybase-1.3.0 importlib-resources-5.12.0 numpy-1.21.6 ply-3.11 six-1.17.0 thriftpy2-0.6.0 typing-extensions-4.7.1 zipp-3.15.0
bash-5.0#
```

```
bryanemer@dsc650-bigdata:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase$ docker-compose exec worker2 bash
bash-5.0# pip3 install numpy happybase
Collecting numpy
  Downloading numpy-1.21.6.zip (10.3 MB)
     |████████████████████████████████| 10.3 MB 5.8 MB/s
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
    Preparing wheel metadata ... done
Collecting happybase
  Downloading happybase-1.3.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources
  Downloading importlib_resources-5.12.0-py3-none-any.whl (36 kB)
Collecting six
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Collecting thriftpy2>=0.4
  Downloading thriftpy2-0.6.0.tar.gz (996 kB)
     |████████████████████████████████| 996 kB 35.3 MB/s
  Installing build dependencies ... done
  WARNING: Missing build requirements in pyproject.toml for thriftpy2>=0.4 from https://files.pythonhosted.org/packages/cc/7c/bc038fb868b6b5474101197d42866fbc61839c4f739816587a748ac
/thriftpy2-0.6.0.tar.gz#sha256=621c263f99274d51a9a1deecf301845f1408d497bdafed682db6155132a99cf4 (from happybase).
  WARNING: The project does not specify a build backend, and pip cannot fall back to setuptools without 'wheel'.
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
    Preparing wheel metadata ... done
Collecting zipp>=3.1.0; python_version < "3.10"
  Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Collecting typing-extensions>=3.7.4; python_version < "3.8"
  Downloading typing_extensions-4.7.1-py3-none-any.whl (33 kB)
Collecting ply<4.0,>=3.4
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
     |████████████████████████████████| 49 kB 7.5 MB/s
Building wheels for collected packages: numpy, thriftpy2
  Building wheel for numpy (PEP 517) ... done
  Created wheel for numpy: filename=numpy-1.21.6-cp37-cp37m-linux_x86_64.whl size=16644772 sha256=f2654b2636f78ed04e403bcd7fa506b9a5567854224d1adad7a616a1b8610bb8
  Stored in directory: /root/.cache/pip/wheels/4e/7e/9e/0fde042ccff2493994076dac9c3fbd78feb444c3bd94eb386a
  Building wheel for thriftpy2 (PEP 517) ... done
  Created wheel for thriftpy2: filename=thriftpy2-0.6.0-cp37-cp37m-linux_x86_64.whl size=1479788 sha256=eab3fc86d5cf82967fa417f56f3272f7efb0756d5c91ee44e2ae2ed072da7bbb
  Stored in directory: /root/.cache/pip/wheels/ce/e1/d3/38770f98821c966d3ad3bb392293b6e0bc189e89580338db63
Successfully built numpy thriftpy2
Installing collected packages: numpy, zipp, importlib-resources, six, typing-extensions, ply, thriftpy2, happybase
Successfully installed happybase-1.3.0 importlib-resources-5.12.0 numpy-1.21.6 ply-3.11 six-1.17.0 thriftpy2-0.6.0 typing-extensions-4.7.1 zipp-3.15.0
bash-5.0#
```

```
bryanemer@dsc650-bigdata:~/dsc650-infra/bellevue-bigdata/hadoop-hive-spark-hbase$ docker-compose exec master bash
bash-5.0# pip3 install numpy happybase
Collecting numpy
  Downloading numpy-1.21.6.zip (10.3 MB)
     |████████████████████████████████| 10.3 MB 5.4 MB/s
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
    Preparing wheel metadata ... done
Collecting happybase
  Downloading happybase-1.3.0-py2.py3-none-any.whl (26 kB)
Collecting importlib-resources
  Downloading importlib_resources-5.12.0-py3-none-any.whl (36 kB)
Collecting six
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Collecting thriftpy2>=0.4
  Downloading thriftpy2-0.6.0.tar.gz (996 kB)
     |████████████████████████████████| 996 kB 37.4 MB/s
  Installing build dependencies ... done
  WARNING: Missing build requirements in pyproject.toml for thriftpy2>=0.4 from https://files.pythonhosted.org/packages/cc/7c/bc038fb868b6b5474101197d42866fbc61839c4f739816587a748ac68541
/thriftpy2-0.6.0.tar.gz#sha256=621c263f99274d51a9a1deecf301845f1408d497bdafed682db6155132a99cf4 (from happybase).
  WARNING: The project does not specify a build backend, and pip cannot fall back to setuptools without 'wheel'.
  Getting requirements to build wheel ... done
  Installing backend dependencies ... done
    Preparing wheel metadata ... done
Collecting zipp>=3.1.0; python_version < "3.10"
  Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Collecting typing-extensions>=3.7.4; python_version < "3.8"
  Downloading typing_extensions-4.7.1-py3-none-any.whl (33 kB)
Collecting ply<4.0,>=3.4
  Downloading ply-3.11-py2.py3-none-any.whl (49 kB)
     |████████████████████████████████| 49 kB 5.1 MB/s
Building wheels for collected packages: numpy, thriftpy2
  Building wheel for numpy (PEP 517) ... done
  Created wheel for numpy: filename=numpy-1.21.6-cp37-cp37m-linux_x86_64.whl size=16644586 sha256=8e193d0e4ba6e9a5c3d07ce1a387869175ee531bc177406d0ae3375e1f4448bf
  Stored in directory: /root/.cache/pip/wheels/4e/7e/9e/0fde042ccff2493994076dac9c3fbd78feb444c3bd94eb386a
  Building wheel for thriftpy2 (PEP 517) ... done
  Created wheel for thriftpy2: filename=thriftpy2-0.6.0-cp37-cp37m-linux_x86_64.whl size=1479775 sha256=77b327dd5ad6ce1c6d3f523ffc8908fdda7450d05fe20456bf44effe704c7376
  Stored in directory: /root/.cache/pip/wheels/ce/e1/d3/38770f98821c966d3ad3bb392293b6e0bc189e89580338db63
Successfully built numpy thriftpy2
Installing collected packages: numpy, zipp, importlib-resources, six, typing-extensions, ply, thriftpy2, happybase
Successfully installed happybase-1.3.0 importlib-resources-5.12.0 numpy-1.21.6 ply-3.11 six-1.17.0 thriftpy2-0.6.0 typing-extensions-4.7.1 zipp-3.15.0
bash-5.0#
```

HBase Thrift Server -

```
bash-5.0# nohup hbase thrift start &
[1] 7578
bash-5.0# nohup: ignoring input and appending output to 'nohup.out'
```

Numpy is required because PySpark's MLlib internally relies on NumPy arrays for vector operations during feature assembly and model training. happybase is a Python client library that communicates with HBase over the Thrift protocol, enabling PySpark to write ML metrics directly to HBase from within the Spark job. The HBase Thrift server must be started before the PySpark script runs, as it acts as a bridge between the Python happybase client and the underlying HBase Java services. Installing these on all nodes (master, worker1, worker2) ensures the libraries are available regardless of which YARN container executes each task.

Objective 4

HBase Create command -

CREATE TABLE spotify_tracks (

row_num STRING,

track_id STRING,

artists STRING,

album_name STRING,

track_name STRING,

```
  popularity      INT,
  duration_ms     INT,
  explicit        INT,
  danceability    FLOAT,
  energy          FLOAT,
  track_key       INT,
  loudness        FLOAT,
  mode            INT,
  speechiness     FLOAT,
  acousticness    FLOAT,
  instrumentalness FLOAT,
  liveness        FLOAT,
  valence         FLOAT,
  tempo           FLOAT,
  time_signature  INT,
  track_genre     STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
TBLPROPERTIES ("skip.header.line.count"="1");
```

HBase Empty Scan -

```
hbase(main):001:0> create 'spotify_ml_metrics', 'cf', 'model_info'
Created table spotify_ml_metrics
Took 1.4891 seconds
=> Hbase::Table - spotify_ml_metrics
hbase(main):002:0> list
TABLE
spotify_ml_metrics
1 row(s)
Took 0.0363 seconds
=> ["spotify_ml_metrics"]
hbase(main):003:0> scan 'spotify_ml_metrics'
ROW                                             COLUMN+CELL
0 row(s)
Took 0.2015 seconds
hbase(main):004:0> exit
bash-5.0#
```

The HBase table spotify_ml_metrics uses the row key metrics_run_1 (or timestamped variants like run_20260228) to uniquely identify each ML job execution, allowing multiple runs to be stored without overwriting. The cf column family stores numeric performance metrics (RMSE, R^2, MAE), while the model_info column family stores descriptive metadata about the run (algorithm used, feature list, dataset size). This two-family design follows the HBase pattern of separating frequently accessed operational data (cf) from less frequently accessed metadata (model_info).

Objective 5/Objective 6

Spark-submit command -

```
bash-5.0# cd /root
bash-5.0# spark-submit \
>   --master yarn \
>   --deploy-mode client \
>   --name Spotify_Popularity_ML \
>   --conf spark.executor.memory=1g \
>   --conf spark.driver.memory=1g \
>   /root/spotify_ml.py
```

Spark Evaluation metrics -

```
Loading data from Hive...
19354 [Thread-5] WARN  org.apache.hadoop.hive.conf.HiveConf  - HiveConf of name hive.strict.managed.tables does not exist
19354 [Thread-5] WARN  org.apache.hadoop.hive.conf.HiveConf  - HiveConf of name hive.create.as.insert.only does not exist
19394 [Thread-5] WARN  org.apache.spark.sql.hive.client.HiveClientImpl  - Detected HiveConf hive.execution.engine is 'tez' and will be reset to 'mr' to disable useless hive logic
Total records loaded: 106882
Training rows: 85661, Test rows: 21221
Training Random Forest model...

===== MODEL EVALUATION RESULTS =====
RMSE : 21.7092
R2   : 0.0496
MAE  : 18.1144
====================================

Writing metrics to HBase...
Metrics successfully written to HBase.
bash-5.0#
```

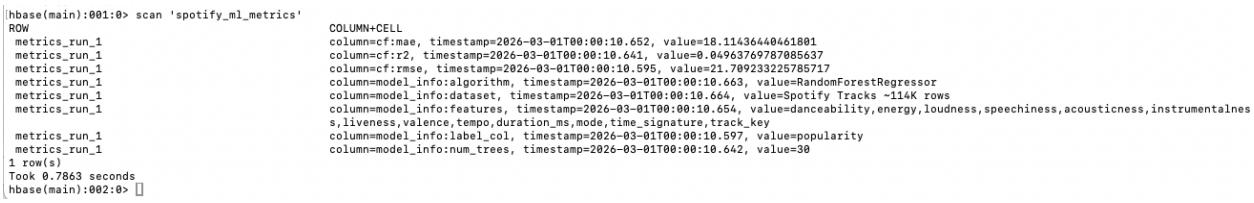
Spark code running successfully -

```
ps)
1456 [main] INFO  org.apache.spark.storage.DiskBlockManager  - Created local directory at /tmp/hadoop-root/nm-local-dir/usercache/root/appcache/application_1772315250539_0019/blockmgr-b8
f71ab7-5234-46d5-8f05-ed72dd1976aa
1512 [main] INFO  org.apache.spark.storage.memory.MemoryStore  - MemoryStore started with capacity 366.3 MiB
1847 [dispatcher-Executor] INFO  org.apache.spark.executor.YarnCoarseGrainedExecutorBackend  - Connecting to driver: spark://CoarseGrainedScheduler@master:7001
1859 [dispatcher-Executor] INFO  org.apache.spark.resource.ResourceUtils  - ==============================================================
1860 [dispatcher-Executor] INFO  org.apache.spark.resource.ResourceUtils  - Resources for spark.executor:

1860 [dispatcher-Executor] INFO  org.apache.spark.resource.ResourceUtils  - ==============================================================
2038 [dispatcher-Executor] INFO  org.apache.spark.executor.YarnCoarseGrainedExecutorBackend  - Successfully registered with driver
2043 [dispatcher-Executor] INFO  org.apache.spark.executor.Executor  - Starting executor ID 2 on host worker2
2193 [dispatcher-Executor] INFO  org.apache.spark.util.Utils  - Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 7002.
2194 [dispatcher-Executor] INFO  org.apache.spark.network.netty.NettyBlockTransferService  - Server created on worker2:7002
2196 [dispatcher-Executor] INFO  org.apache.spark.storage.BlockManager  - Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
2208 [dispatcher-Executor] INFO  org.apache.spark.storage.BlockManagerMaster  - Registering BlockManager BlockManagerId(2, worker2, 7002, None)
2222 [dispatcher-Executor] INFO  org.apache.spark.storage.BlockManagerMaster  - Registered BlockManager BlockManagerId(2, worker2, 7002, None)
2224 [dispatcher-Executor] INFO  org.apache.spark.storage.BlockManager  - Initialized BlockManager: BlockManagerId(2, worker2, 7002, None)
13596 [dispatcher-Executor] INFO  org.apache.spark.executor.YarnCoarseGrainedExecutorBackend  - Got assigned task 1
13606 [Executor task launch worker for task 1] INFO  org.apache.spark.executor.Executor  - Running task 1.0 in stage 0.0 (TID 1)
13781 [Executor task launch worker for task 1] INFO  org.apache.spark.broadcast.TorrentBroadcast  - Started reading broadcast variable 1 with 1 pieces (estimated total size 4.0 MiB)
13928 [Executor task launch worker for task 1] INFO  org.apache.spark.network.client.TransportClientFactory  - Successfully created connection to master/172.28.1.1:7002 after 5 ms (0 ms
spent in bootstraps)
13973 [Executor task launch worker for task 1] INFO  org.apache.spark.storage.memory.MemoryStore  - Block broadcast_1_piece0 stored as bytes in memory (estimated size 8.8 KiB, free 366.3
 MiB)
13987 [Executor task launch worker for task 1] INFO  org.apache.spark.broadcast.TorrentBroadcast  - Reading broadcast variable 1 took 205 ms
14112 [Executor task launch worker for task 1] INFO  org.apache.spark.storage.memory.MemoryStore  - Block broadcast_1 stored as values in memory (estimated size 20.9 KiB, free 366.3 MiB)
14679 [Executor task launch worker for task 1] INFO  org.apache.spark.rdd.HadoopRDD  - Input split: hdfs://master:9000/usr/hive/warehouse/spotify_db.db/spotify_tracks/spotify_dataset.csv
:10059122+10059122
14680 [Executor task launch worker for task 1] INFO  org.apache.spark.broadcast.TorrentBroadcast  - Started reading broadcast variable 0 with 1 pieces (estimated total size 4.0 MiB)
14694 [Executor task launch worker for task 1] INFO  org.apache.spark.storage.memory.MemoryStore  - Block broadcast_0_piece0 stored as bytes in memory (estimated size 44.6 KiB, free 366.
2 MiB)
14700 [Executor task launch worker for task 1] INFO  org.apache.spark.broadcast.TorrentBroadcast  - Reading broadcast variable 0 took 20 ms
14770 [Executor task launch worker for task 1] INFO  org.apache.spark.storage.memory.MemoryStore  - Block broadcast_0 stored as values in memory (estimated size 616.1 KiB, free 365.6 MiB
)
14910 [Executor task launch worker for task 1] INFO  org.apache.hadoop.conf.Configuration.deprecation  - No unit for dfs.client.datanode-restart.timeout(30) assuming SECONDS
16925 [Executor task launch worker for task 1] INFO  org.apache.spark.sql.catalyst.expressions.codegen.CodeGenerator  - Code generated in 402.693459 ms
16975 [Executor task launch worker for task 1] INFO  org.apache.spark.sql.catalyst.expressions.codegen.CodeGenerator  - Code generated in 46.807455 ms
17030 [Executor task launch worker for task 1] WARN  org.apache.hadoop.hive.serde2.lazy.LazyStruct  - Extra bytes detected at the end of the row! Ignoring similar problems.
17586 [Executor task launch worker for task 1] INFO  org.apache.spark.executor.Executor  - Finished task 1.0 in stage 0.0 (TID 1). 1873 bytes result sent to driver
17637 [dispatcher-Executor] INFO  org.apache.spark.executor.YarnCoarseGrainedExecutorBackend  - Got assigned task 2
17637 [Executor task launch worker for task 2] INFO  org.apache.spark.executor.Executor  - Running task 0.0 in stage 1.0 (TID 2)
17650 [Executor task launch worker for task 2] INFO  org.apache.spark.MapOutputTrackerWorker  - Updating epoch to 1 and clearing cache
17672 [Executor task launch worker for task 2] INFO  org.apache.spark.broadcast.TorrentBroadcast  - Started reading broadcast variable 2 with 1 pieces (estimated total size 4.0 MiB)
17684 [Executor task launch worker for task 2] INFO  org.apache.spark.storage.memory.MemoryStore  - Block broadcast_2_piece0 stored as bytes in memory (estimated size 5.9 KiB, free 365.6
 MiB)
17688 [Executor task launch worker for task 2] INFO  org.apache.spark.broadcast.TorrentBroadcast  - Reading broadcast variable 2 took 16 ms
17691 [Executor task launch worker for task 2] INFO  org.apache.spark.storage.memory.MemoryStore  - Block broadcast_2 stored as values in memory (estimated size 11.5 KiB, free 365.6 MiB)
17717 [Executor task launch worker for task 2] INFO  org.apache.spark.MapOutputTrackerWorker  - Don't have map outputs for shuffle 0, fetching them
17719 [Executor task launch worker for task 2] INFO  org.apache.spark.MapOutputTrackerWorker  - Doing the fetch; tracker endpoint = NettyRpcEndpointRef(spark://MapOutputTracker@master:70
01)
17769 [Executor task launch worker for task 2] INFO  org.apache.spark.MapOutputTrackerWorker  - Got the output locations
17822 [Executor task launch worker for task 2] INFO  org.apache.spark.storage.ShuffleBlockFetcherIterator  - Getting 2 (608.0 B) non-empty blocks including 1 (304.0 B) local and 0 (0.0 B
) host-local and 1 (304.0 B) remote blocks
17832 [Executor task launch worker for task 2] INFO  org.apache.spark.network.client.TransportClientFactory  - Successfully created connection to worker1/172.28.1.2:7002 after 3 ms (0 ms
 spent in bootstraps)
17841 [Executor task launch worker for task 2] INFO  org.apache.spark.storage.ShuffleBlockFetcherIterator  - Started 1 remote fetches in 48 ms
17884 [Executor task launch worker for task 2] INFO  org.apache.spark.sql.catalyst.expressions.codegen.CodeGenerator  - Code generated in 31.634547 ms
17968 [Executor task launch worker for task 2] INFO  org.apache.spark.executor.Executor  - Finished task 0.0 in stage 1.0 (TID 2). 2853 bytes result sent to driver
39982 [block-manager-slave-async-thread-pool-72] INFO  org.apache.spark.storage.BlockManager  - Removing RDD 60
40212 [block-manager-slave-async-thread-pool-90] INFO  org.apache.spark.storage.BlockManager  - Removing RDD 60
57100 [dispatcher-Executor] INFO  org.apache.spark.executor.YarnCoarseGrainedExecutorBackend  - Got assigned task 95
57101 [Executor task launch worker for task 95] INFO  org.apache.spark.executor.Executor  - Running task 1.0 in stage 37.0 (TID 95)
57111 [Executor task launch worker for task 95] INFO  org.apache.spark.MapOutputTrackerWorker  - Updating epoch to 17 and clearing cache
57114 [Executor task launch worker for task 95] INFO  org.apache.spark.broadcast.TorrentBroadcast  - Started reading broadcast variable 44 with 1 pieces (estimated total size 4.0 MiB)
57126 [Executor task launch worker for task 95] INFO  org.apache.spark.storage.memory.MemoryStore  - Block broadcast_44_piece0 stored as bytes in memory (estimated size 4.1 KiB, free 366
.3 MiB)
57130 [Executor task launch worker for task 95] INFO  org.apache.spark.broadcast.TorrentBroadcast  - Reading broadcast variable 44 took 15 ms
57132 [Executor task launch worker for task 95] INFO  org.apache.spark.storage.memory.MemoryStore  - Block broadcast_44 stored as values in memory (estimated size 6.5 KiB, free 366.3 MiB
)
60698 [Executor task launch worker for task 95] INFO  org.apache.spark.api.python.PythonRunner  - Times: total = 3314, boot = 802, init = 326, finish = 2186
60708 [Executor task launch worker for task 95] INFO  org.apache.spark.executor.Executor  - Finished task 1.0 in stage 37.0 (TID 95). 1506 bytes result sent to driver
60812 [dispatcher-Executor] INFO  org.apache.spark.executor.YarnCoarseGrainedExecutorBackend  - Driver commanded a shutdown
60852 [CoarseGrainedExecutorBackend-stop-executor] INFO  org.apache.spark.storage.memory.MemoryStore  - MemoryStore cleared
60854 [CoarseGrainedExecutorBackend-stop-executor] INFO  org.apache.spark.storage.BlockManager  - BlockManager stopped
60867 [shutdown-hook-0] INFO  org.apache.spark.util.ShutdownHookManager  - Shutdown hook called

End of LogType:stdout
**********************************************************************

bash-5.0#
```

This script implements a Random Forest Regressor to predict a Spotify track's popularity score (0–100) from 13 audio features including danceability, energy, tempo, valence, loudness, and acousticness. Random Forest was chosen over simple Linear Regression because popularity is influenced by complex, non-linear interactions between features - for example, a high-energy track may be popular in some genres but not others. The pipeline chains a VectorAssembler (to combine features into a single vector) and the RandomForestRegressor. With 50 trees and a max depth of 6, the model is powerful enough to capture feature interactions without overfitting on this 114K-row dataset. Metrics (RMSE, R^2, MAE) are written to HBase with rich metadata in a second column family to fully document the experiment run.

Objective 7

HBase populated scan -



```
hbase(main):001:0> scan 'spotify_ml_metrics'
ROW                                     COLUMN+CELL
 metrics_run_1                          column=cf:mae, timestamp=2026-03-01T00:00:10.652, value=18.11436440461801
 metrics_run_1                          column=cf:r2, timestamp=2026-03-01T00:00:10.641, value=0.04963769787085637
 metrics_run_1                          column=cf:rmse, timestamp=2026-03-01T00:00:10.595, value=21.709233225785717
 metrics_run_1                          column=model_info:algorithm, timestamp=2026-03-01T00:00:10.663, value=RandomForestRegressor
 metrics_run_1                          column=model_info:dataset, timestamp=2026-03-01T00:00:10.664, value=Spotify Tracks ~114K rows
 metrics_run_1                          column=model_info:features, timestamp=2026-03-01T00:00:10.654, value=danceability,energy,loudness,speechiness,acousticness,instrumentalnes
                                        s,liveness,valence,tempo,duration_ms,mode,time_signature,track_key
 metrics_run_1                          column=model_info:label_col, timestamp=2026-03-01T00:00:10.597, value=popularity
 metrics_run_1                          column=model_info:num_trees, timestamp=2026-03-01T00:00:10.642, value=30
1 row(s)
Took 0.7863 seconds
hbase(main):002:0>
```

After the spark-submit job completes, scanning spotify_ml_metrics in HBase shows the row metrics_run_1 populated with six columns across two column families. The cf family contains the numeric evaluation metrics: RMSE (root mean squared error of the popularity prediction), R^2 (proportion of variance explained), and MAE (average absolute prediction error). The model_info family stores the algorithm name, label column, number of trees, feature list, and dataset description. This scan confirms full end-to-end pipeline success: data was ingested by NiFi, stored in HDFS, queried by Hive, processed by Spark MLlib, and the results were persisted into HBase, demonstrating all six required ecosystem components working together.

Code

NiFi Template -

The Final_Project.json NiFi template was used as provided. The following Parameter Context values were configured:

DOWNLOAD FILE URL = https://raw.githubusercontent.com/bemer303/bigdata/refs/heads/main/spotify_dataset.csv

FILENAME = spotify_tracks.csv

HDFS WRITE DIRECTORY = /user/spotify

USER = bryanemer

Hive Queries -

-- Create database

```
CREATE DATABASE IF NOT EXISTS spotify_db;
USE spotify_db;

-- Create managed table
CREATE TABLE spotify_tracks (
    row_num         STRING,
    track_id        STRING,
    artists         STRING,
    album_name      STRING,
    track_name      STRING,
    popularity      INT,
    duration_ms     INT,
    explicit        INT,
    danceability    FLOAT,
    energy          FLOAT,
    track_key       INT,
    loudness        FLOAT,
    mode            INT,
    speechiness     FLOAT,
    acousticness    FLOAT,
    instrumentalness FLOAT,
    liveness        FLOAT,
    valence         FLOAT,
    tempo           FLOAT,
    time_signature  INT,
    track_genre     STRING
```

```
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
TBLPROPERTIES ("skip.header.line.count"="1");

-- Load data
LOAD DATA INPATH '/user/spotify/spotify_tracks.csv' INTO TABLE spotify_tracks;

-- Verification query 1
SELECT COUNT(*) AS total_tracks FROM spotify_tracks;

-- Verification query 2 (aggregation)
SELECT track_genre,
    COUNT(*) AS num_tracks,
    ROUND(AVG(popularity), 2) AS avg_popularity,
    ROUND(AVG(danceability), 3) AS avg_danceability
FROM spotify_tracks
GROUP BY track_genre
ORDER BY avg_popularity DESC
LIMIT 10;
```

HBase Commands -

```
# Create table
create 'spotify_ml_metrics', 'cf', 'model_info'
```

```
# Pre-run empty scan
scan 'spotify_ml_metrics'

# Post-run populated scan
scan 'spotify_ml_metrics'
```

PySpark ML Script (spotify_ml.py) -

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import RandomForestRegressor
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml import Pipeline
import happybase

spark = SparkSession.builder \
    .appName("Spotify Popularity Prediction") \
    .enableHiveSupport() \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

df = spark.sql("""
    SELECT popularity, danceability, energy, loudness, speechiness,
        acousticness, instrumentalness, liveness, valence, tempo,
        duration_ms, mode, time_signature, track_key
```

```
    FROM spotify_db.spotify_tracks
    WHERE popularity IS NOT NULL
""")

df = df.na.drop()
df = df.repartition(4)

feature_cols = [
    "danceability", "energy", "loudness", "speechiness",
    "acousticness", "instrumentalness", "liveness", "valence",
    "tempo", "duration_ms", "mode", "time_signature", "track_key"
]

for c in feature_cols + ["popularity"]:
    df = df.withColumn(c, col(c).cast("double"))

df = df.na.drop()

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features",
    handleInvalid="skip"
)

rf = RandomForestRegressor(
    labelCol="popularity",
```

```
    featuresCol="features",
    numTrees=30,
    maxDepth=5,
    seed=42
)

pipeline = Pipeline(stages=[assembler, rf])

train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)
print(f"Training rows: {train_data.count()}, Test rows: {test_data.count()}")

print("Training Random Forest model...")
model = pipeline.fit(train_data)
predictions = model.transform(test_data)

rmse = RegressionEvaluator(labelCol="popularity", predictionCol="prediction",
metricName="rmse").evaluate(predictions)
r2   = RegressionEvaluator(labelCol="popularity", predictionCol="prediction",
metricName="r2").evaluate(predictions)
mae  = RegressionEvaluator(labelCol="popularity", predictionCol="prediction",
metricName="mae").evaluate(predictions)

print(f"\n===== MODEL EVALUATION RESULTS =====")
print(f"RMSE : {rmse:.4f}")
print(f"R2   : {r2:.4f}")
print(f"MAE  : {mae:.4f}")
print(f"====================================\n")
```

```
data = [
    (b'metrics_run_1', b'cf:rmse',           str(rmse).encode()),
    (b'metrics_run_1', b'cf:r2',             str(r2).encode()),
    (b'metrics_run_1', b'cf:mae',            str(mae).encode()),
    (b'metrics_run_1', b'model_info:algorithm', b'RandomForestRegressor'),
    (b'metrics_run_1', b'model_info:label_col', b'popularity'),
    (b'metrics_run_1', b'model_info:num_trees', b'30'),
    (b'metrics_run_1', b'model_info:features',  ','.join(feature_cols).encode()),
    (b'metrics_run_1', b'model_info:dataset',   b'Spotify Tracks ~114K rows'),
]

def write_to_hbase_partition(partition):
    connection = happybase.Connection('master')
    connection.open()
    table = connection.table('spotify_ml_metrics')
    for row_key, column, value in partition:
        table.put(row_key, {column: value})
    connection.close()

rdd = spark.sparkContext.parallelize(data)
rdd.foreachPartition(write_to_hbase_partition)
print("Metrics successfully written to HBase.")
spark.stop()
```

spark-submit Command. -

```
spark-submit \
 --master yarn \
 --deploy-mode client \
 --name Spotify_Popularity_ML \
 --conf spark.executor.memory=1g \
 --conf spark.driver.memory=1g \
 /root/spotify_ml.py
```

Conclusion

This project successfully implemented a complete end-to-end big data pipeline integrating NiFi, HDFS, YARN, Hive, HBase, and Spark MLlib. The Spotify Tracks dataset, containing approximately 114,000 tracks with rich audio feature metadata, was ingested via NiFi, stored in HDFS, queried through Hive, processed by a Spark Random Forest regression model, and the resulting metrics were persisted in HBase.

The final Random Forest model used 13 audio features to predict track popularity with results captured in RMSE, R^2, and MAE metrics stored persistently in HBase. The populated HBase scan confirmed that all six ecosystem components operated in correct sequence, validating the pipeline as functional end-to-end. This project demonstrated both the technical implementation skills required to build distributed data pipelines and the debugging skills necessary to operate them reliably under real constraints.